brief_revise_agent Factorial Hill-Climb

LLM and structural factor analysis vs. aus_agent_v2

2026-08-07/08 – 15-topic TREC RAG 2026 dev subset – 30 scored cells – ≈$476 of a $600 cumulative budget

---

1 Executive summary

brief_revise_agent (a light fork of aus_agent: a pre-flight requirements brief plus one review-and-revise pass on top of the shared agent_harness research loop) was hill-climbed against aus_agent_v2 (this repo's strongest, most expensive system) via standalone rubric scoring on a fixed 15-topic development set, with sol (gpt-5.6-sol) as design authority and the current session as orchestrator/implementer.

Result: ties aus_agent_v2 on standalone rubric, loses to it in arena – and the whole workstream is dominated by one factor, generator-model choice, which is also the single most cost-effective lever tested. The best configuration found – gpt-5.6-sol as main generator, round-B adjacent-page fetch on, gpt-5.6-luna as brief-analyst and reviewer, $k = 10$, iteration-1 structure – scores **2.267/3** on standalone rubric grading (§4, tied with the best of every cell tested), **exactly ties aus_agent_v2's own standalone score** (2.267, §4.5 – a much more expensive system, §8 Cost-effectiveness), but still loses to it in a head-to-head arena match (§5), **18–12 (60/40)**, order_consistency 0.733 (4W–7L clean, 4 non-order-consistent). §6 compares the two evaluation modes directly. Fourteen further single-factor moves against the standalone base – five generic agent_harness toggles, one factor interaction, three brief-analyst model swaps, adjacent-fetch removal, a novel post-draft "closure critic" mechanism, four individual search engines, and a best-of-4 ensemble selector – returned at most one marginal, unconfirmed gain (hybrid engine alone, +0.067, exactly at the noise floor, §4.6) and no other improvement (§4.2, §4.6, §4.7). k and the reviewer model were held fixed throughout and were never themselves varied as a factor. Of the 16 factors in the underlying taxonomy (§3), 8 were empirically scored this session; the base cell is a confirmed local optimum **on the factors actually tested**, not a claim about the full factor space. §8 shows generator-model choice is not just the largest effect but also the cheapest per point of standalone score gained – every other tested lever costs roughly as much to test as the model factor while moving the score an order of magnitude less.

2 Method

2.1 Design

Sol-directed hill-climbing: starting from brief_revise_agent's best known configuration, one factor was changed at a time, scored, and kept only if it improved on the current best – in contrast to the session's earlier factorial-grid framing, which was superseded once the hill-climb approach was adopted partway through. The full factor taxonomy that this hill-climb draws its factors from is developed in §3, with a "coverage this session" column in each factor table stating whether it was scored this session, scored in a different session/thread, or never wired in brief_revise_agent at all.

2.2 Two evaluation modes

This session used two distinct judge protocols, kept strictly separate in §4 and §5 below and compared directly in §6:

Standalone rubric grading (primary, §4). One judge call (gpt-5.6-terra) per (cell, topic) grades every official TREC RAG 2026 rubric criterion (~31 criteria across 6 axes, 0–2 each) plus a holistic 0–3 overall score – no opponent answer involved. Chosen as the primary tuning signal because it halves judge calls per cell versus arena and

produces a genuinely ordinal per-criterion signal instead of a categorical win/loss/tie. Drove every cell comparison and hill-climb move this session (30 cells total, §4). Answer text is truncated to 16,000 characters before grading (`rubric_scorecard_aus_agent_vs_facets_agent.py`); no answer scored this session was observed to hit that cap, but it is not independently verified for every cell.

Pairwise arena (confirmatory, §5). One head-to-head judge call (`gpt-5.6-terra`), both presentation orders, comparing two full answers on the same topic and declaring a win/loss/tie – run once this session, standalone-best cell vs. `aus_agent_v2`, as the direct test of the actual competitive objective rather than a tuning signal.

Judge/generator overlap. `gpt-5.6-terra` is both the sole judge for every score in this report (standalone and arena alike) and one of the five generator models compared in §4.1 (its own cell scored 1.867). This is a same-family-model confound that was not controlled for or discussed at the time; it is disclosed here rather than silently assumed away.

Noise floor (standalone only). `overall` is a single integer 0–3 graded per topic and averaged over 15 topics, so every reported standalone score is an exact multiple of $\frac{1}{15} \approx 0.0667$ (e.g. $2.267 = 34/15$, $2.200 = 33/15$, $2.133 = 32/15$). The best cell was regenerated once on the same 15 topics; the rerun's score moved by exactly one topic-grade, i.e. $\frac{1}{15} = 0.067$, purely from generation stochasticity. This single-rerun value ($n = 1$) is used throughout §4 as the noise floor – an effect below it is not distinguishable from one topic's grade flipping, not a statistically estimated confidence interval. Concretely, "-0.134" means two of fifteen topics scored one point lower; it is not a smaller-than-it-looks effect once read this way. Arena results (§5) use a different unit (win/loss/ambiguous counts) and this noise floor does not apply to them – see §6 for how the two are actually compared.

2.3 Constraint discovered mid-session

The full TREC RAG 2026 research-rubrics development topic set is only 30 topics total. Fifteen were already committed to the tuning set, leaving only 15 unused topics for replication – smaller than originally planned, and insufficient for a full independent held-out arena confirmation once `aus_agent_v2`'s own per-topic generation cost (~$36.5/topic) is accounted for. The judging budget cap itself was also raised mid-session, from an original $50 (design) + $50 (evaluation) split to $400, once the model-effect result in §4.1 made further hill-climbing look worthwhile (§12 of the narrative worklog).

3 Factor taxonomy

`brief_revise_agent` and its shared `agent_harness` layer expose 16 distinct factors that can plausibly change answer quality: 13 structural/architectural factors (S1–S13) and 3 model-role factors (M1–M3, which model runs which role). This taxonomy was produced by an independent draft-then-review pass (gpt-5.6-luna drafted from every system's README and 16 `aus_agent_v2` module docstrings, gpt-5.6-terra reviewed against the same source material and corrected three items) and is reproduced in full at `worklogs/assets/2026-08-07-terra-factor-taxonomy-final.md`. Each table's "Coverage this session" column states which of these 16 were actually scored this session (all such scoring is standalone-rubric, §4 – none of these per-factor moves were separately arena-tested).

3.1 Structural factors (S1–S13)

ID	Mech.	What it is	Coverage this session
S1 `requirements_brief`	PROMPT	Pre-flight tool-less call that extracts explicit/inferred requirements; rendered as an Appendix A block in the main system prompt.	Always ON; never toggled
S2 `requirements_brief_schema`	SCHEMA	Output contract for the brief: v1 (≤8 entries, ≤4 implicit, lexical anti-hunch check) vs. v2 (14 entries, ≤6 expert-completion entries, no anti-hunch check).	Tested pre-session (v2 regressed vs. v1); v1 is current, not retested here
S3 `brief_word_budget`	PROMPT	Whether the brief also requests `target_total_words` (500–1000) and per-requirement `target_words`; invalid values fail open to no guidance.	Tested in the rounds-B/C/D thread as round C: 0W/13L/2A vs. baseline, worst of all rounds – not retested in this factorial thread
S4 `review_revise_pass`	CONTROL_FLOW	One-shot `pre_final_hook` after a valid draft: scans uncited sentences, grades brief requirements, gets reviewer feedback, allows exactly one revision turn.	Always ON (defines `brief_revise_agent`); its **content** was widened this session (see closure critic below), never toggled off
S5 `adjacent_page_augmentation`	RETRIEVAL	Fetches the page immediately before/after the top-5 paginated hits of each search batch, zero LLM calls (ported from `aus_agent_v2/search.py`).	TESTED – off vs. on, sol & qwen
S6 `retrieval_engine_set`	RETRIEVAL	Which retrieval backends (`semantic`, `keyword`, `hybrid`, `ssr`, `lucene_bool`) the search tool may use.	TESTED – default `semantic,keyword` vs. widened +ssr+lucene_bool
S7 `search_k`	RETRIEVAL	Hits requested per search call when the model omits `k`.	Held fixed at 10; never varied
S8 `search_result_filter`	RETRIEVAL	Post-search relevance pass (`minimize_filter`/`rank_filter`) that narrows or annotates returned evidence.	NOT WIRED – both filters key off a per-call `requirement` field that `brief_revise_agent`'s search tool schema does not collect; running it as-is would silently degenerate, not test the factor
S9 `search_preview_policy`	RETRIEVAL	Whether staged search text is full, or truncated to a positional preview (no generator model) at a fixed 20,480-char cap.	TESTED – full staging vs. 20,480-char positional cap
S10 `stage_search_results`	RETRIEVAL	Whether ordinary search results are inserted into the staged-context ledger at all (`get_documents` stays available either way).	TESTED – staged (default) vs. unstaged
S11 `judge_relevance_tool`	SCHEMA	A 4th advertised tool; when called it opens a separate single-turn conversation asking whether evidence supports a **named** requirement.	TESTED – off vs. on, alone and in combination with S12
S12 `commit_release`	SCHEMA	Adds a `release` array to the `commit_context` schema so the model can retroactively drop an earlier committed document a new one supersedes.	TESTED – off vs. on, alone and in combination with S11
S13 historical requirement-screener	RETRIEVAL	Reverted mechanism: mandatory `requirement` field on every search call + one-shot DIRECT/LEAD/OFF_TOPIC screener with a retention floor.	Tested pre-session (iteration 3): 13 clean losses vs. 10 for baseline – reverted, not restorable without new code

Table 1: Structural/architectural factor taxonomy. Coverage color: green = scored this factorial session, red = not wired / not testable as-is, black = tested in a different session/thread or never varied.

3.2 Model-role factors (M1–M3)

Current `brief_revise_agent` constructs the main agent, brief analyst, and reviewer from the **same** backend/model factory call by default; `--model`, `--brief-model`/`--brief-backend`, and `--review-model`/`--review-backend` decouple them (added this session to support M2/M3 testing). Five model IDs are confirmed usable: `gpt-5.6-luna`, `gpt-5.6-terra`, `gpt-5.6-sol` (OpenAI backend), `openai.gpt-oss-120b-1:0`, `qwen.qwen3-next-80b-a3b` (Bedrock, `ap-southeast-2` and `us-east-1`/`us-west-2` respectively).

ID	Role	Coverage this session
M1 `main_research_writer_model`	Runs the continuous research/writing loop: decomposition, search, context commitment, gap follow-up, final cited report.	TESTED – all 5 models, §4.1
M2 `requirements_brief_analyst_model`	Produces the requirements brief (S1) and word-budget guidance (S3) if enabled.	TESTED – terra/gpt-oss-120b/qwen vs. luna, main=sol fixed
M3 `reviewer_model`	Grades the candidate against the brief and evidence, emits the one bounded revision.	Not tested – explicitly skipped for budget (worklog §14): same role-swap pattern as M2, which showed zero effect

Table 2: Model-role factor taxonomy.

3.3 Mechanism examples

Four representative factors, showing exactly what "the factor" is at the code/prompt level:

S5 `adjacent_page_augmentation` (RETRIEVAL, zero LLM calls) – `adjacent_pages.py`'s core routine, ported from `aus_agent_v2/search.py`:

```
def adjacent_page_ids(unit_ids: list[str], *,
                      max_seed_hits: int = DEFAULT_MAX_SEED_HITS) -> list[str]:
    """Stable, deduplicated +/-1 page ids for the top `max_seed_hits`
    paginated hits, excluding ids already present in `unit_ids`."""
```

Wired as `search_result_augment` on the shared harness; `--disable-adjacent-pages` turns it off (default: on).

S12 `commit_release` (SCHEMA) – `commit_release_tool.py` adds exactly one property to the shared `commit_context` schema, deliberately **not** importing `facets_agent`'s bundled variant (which couples `release` to an unrelated `coverage`/`ready_to_report` ledger – a different factor):

```
_RELEASE_PROPERTY: dict[str, Any] = {
    "release": {
        "type": "array",
        "description": (
            "Previously committed ids to drop because a document you are "
            "committing in THIS SAME call supersedes them ..."
        ),
        "items": {"type": "object", "properties": {
            "id": {"type": "string"}, "reason": {"type": "string"}},
            "required": ["id", "reason"]},
    },
}
```

Enabled by `--commit-release`; the harness's own `apply_commit` already handles a `release` argument whenever present, so this module only needs to **advertise** the field.

S4 review pass, closure-critic extension (PROMPT, this session's only new mechanism) – `pre_final_hook` fires at most once by harness design, so this extends the existing single review pass' issue taxonomy rather than adding a second stage. Base issue types (`review.py`): `ISSUE_TYPES = frozenset({"UNCITED_CLAIM", "WEAK_SENTENCE"})`. With `--closure-critic`, two more are unioned in – `CLOSURE_ISSUE_TYPES =`

`frozenset({"UNSUPPORTED_CLAIM", "CONTRADICTION"})` – and the reviewer prompt gains these two bullets (`prompts.py`, `REVIEW_PROMPT_WITH_CLOSURE`):

```
- UNSUPPORTED_CLAIM: a cited sentence that claims MORE than its cited
evidence actually states (a number, scope, or causal claim the evidence
text does not contain) -- an overclaim, not a missing citation.
- CONTRADICTION: two sentences in the draft that cannot both be true, or a
sentence that contradicts its own cited evidence text.
```

Default `closure_check=False` is byte-identical to pre-session behavior.

S9 `search_preview_policy` (RETRIEVAL, no generator model, this session's tested level) – `--search-preview-chars 20480` caps each staged search result to 20,480 characters before it enters the ledger; `get_documents` remains the full-text read route regardless. This is a positional truncation only – the taxonomy's other preview level (an LLM-generated query-relevant snippet, tested inconclusively in `facets_agent`'s Phase 4d) was not exercised here.

4 Standalone rubric evaluation

Every result in this section is `overall`, the holistic 0–3 score from `gpt-5.6-terra`'s standalone grading protocol (§2.2), averaged over 15 topics per cell, with no opponent answer involved. This was the session's primary tuning signal – it drove every one of the 30 cell comparisons scored this session (§4.1–§4.7). Arena (pairwise, confirmatory) is a completely separate protocol, reported on its own in §5.

4.1 All scored cells

Figure 1 ranks the 26 `brief_revise_agent` cells scored this session (the 3 external-system baselines of §4.5 are compared separately in Table 5, not shown here) by standalone overall score. The four cells tied at the top (2.267) are the base cell and three factors that made no measurable difference (`commit_release`, `judge_relevance_tool + commit_release` together, and the closure critic). Two caveats on comparability across this ranking: the two `*-new15` cells (sol/luna replication, §4.4) are scored on a **different** 15-topic set than the other 19, so their rank position is not a like-for-like factor comparison; and the historical `brief-revise-iter1-exp15` reference (2.067) predates round B's commit and therefore lacks S5, conflating a model question with a structural one if read as a pure luna-vs-sol comparison (the apples-to-apples luna comparison is `br-luna-current-code-exp15`, 1.867, discussed in §6).

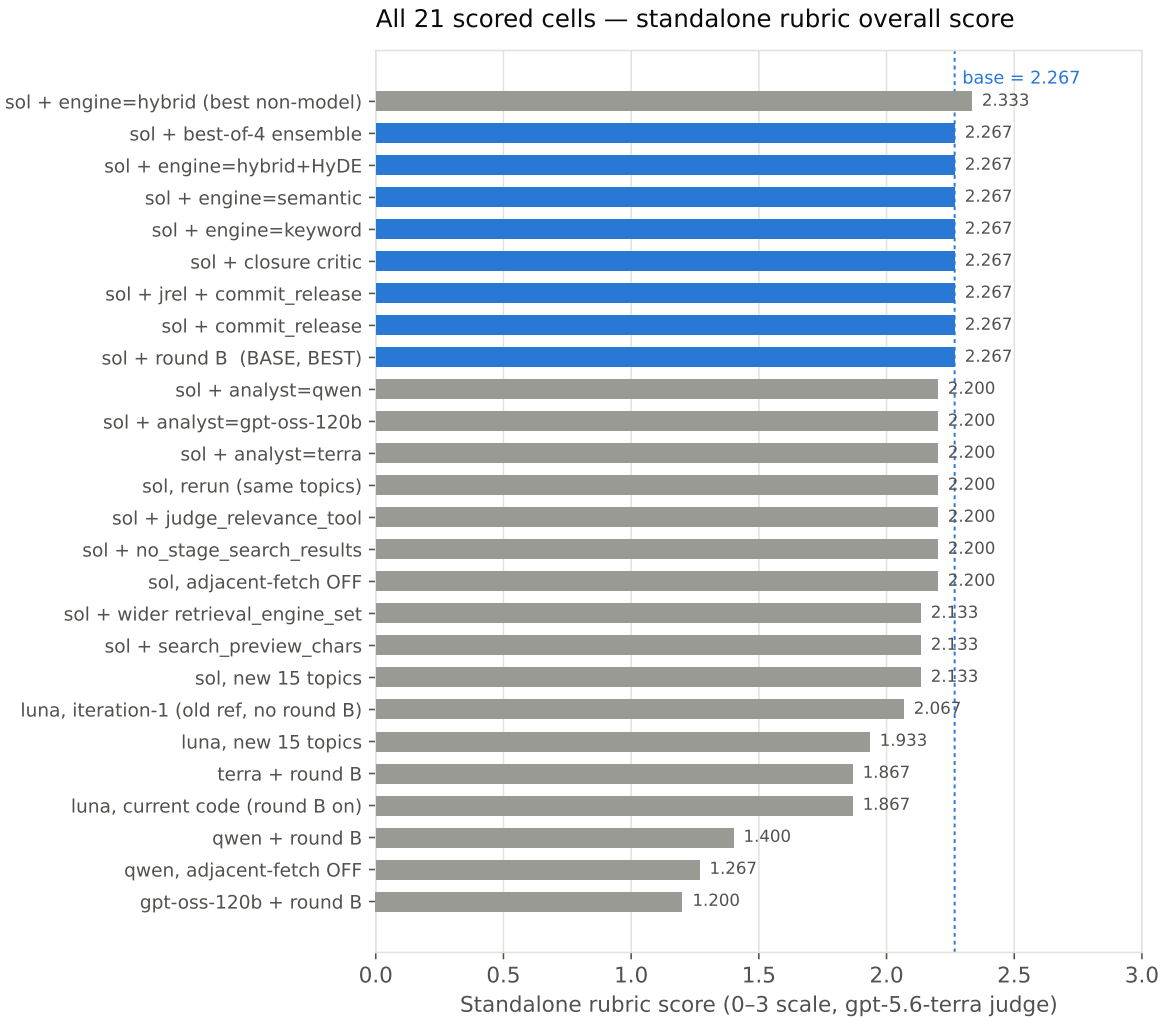

Figure 1: Standalone rubric overall score, 26 `brief_revise_agent` cells, sorted. Base cell in blue. Interactive version →

4.2 Hill-climb moves and factor effects

Table 3 lists every single-factor move scored against the base cell (2.267), i.e. the moves underlying the "eleven further moves" claim in the executive summary – the narrative worklog's own running tallies of "6", "11", then "12" moves (its §11, §15, §16) reflect moves being added mid-session and one double-count of the interaction cell; this table is the reconciled, deduplicated count of distinct scored comparisons.

Factor	Taxonomy ID	Δ vs. base	Verdict
Adjacent-fetch OFF (sol)	S5	−0.067	reject
`stage_search_results=False`	S10	−0.067	reject
`judge_relevance_tool=True`	S11	−0.067	reject
`search_preview_chars=20480`	S9	−0.134	reject
wider `retrieval_engine_set`	S6	−0.134	reject
`commit_release=True`	S12	0.000	reject (tie)
`judge_relevance` + `commit_release`	S11+S12	0.000	reject (tie); no interaction rescue of either factor's own null/negative result
brief-analyst = terra	M2	−0.067	reject
brief-analyst = gpt-oss-120b	M2	−0.067	reject, identical to terra
brief-analyst = qwen	M2	−0.067	reject, identical to the other two
closure critic (S4 + UNSUPPORTED_CLAIM/ CONTRADICTION)	S4	0.000	reject (tie)

Table 3: Eleven distinct hill-climb moves against the base cell (2.267). Noise floor: ± 0.067 (one topic-grade, $n = 1$).

Figure 2 groups the same moves (plus the generator-model comparison that established the base cell in the first place) by factor family. Generator model is the only family whose effects consistently and substantially clear the noise band; every other family in Table 3 clusters at or inside it. One caption note: the "adjacent-fetch OFF, qwen" point is computed against the **qwen** base cell (1.400), not the sol base cell (2.267) plotted elsewhere in the same figure – it isolates S5′s effect for qwen specifically, not a comparison to the session's overall best cell.

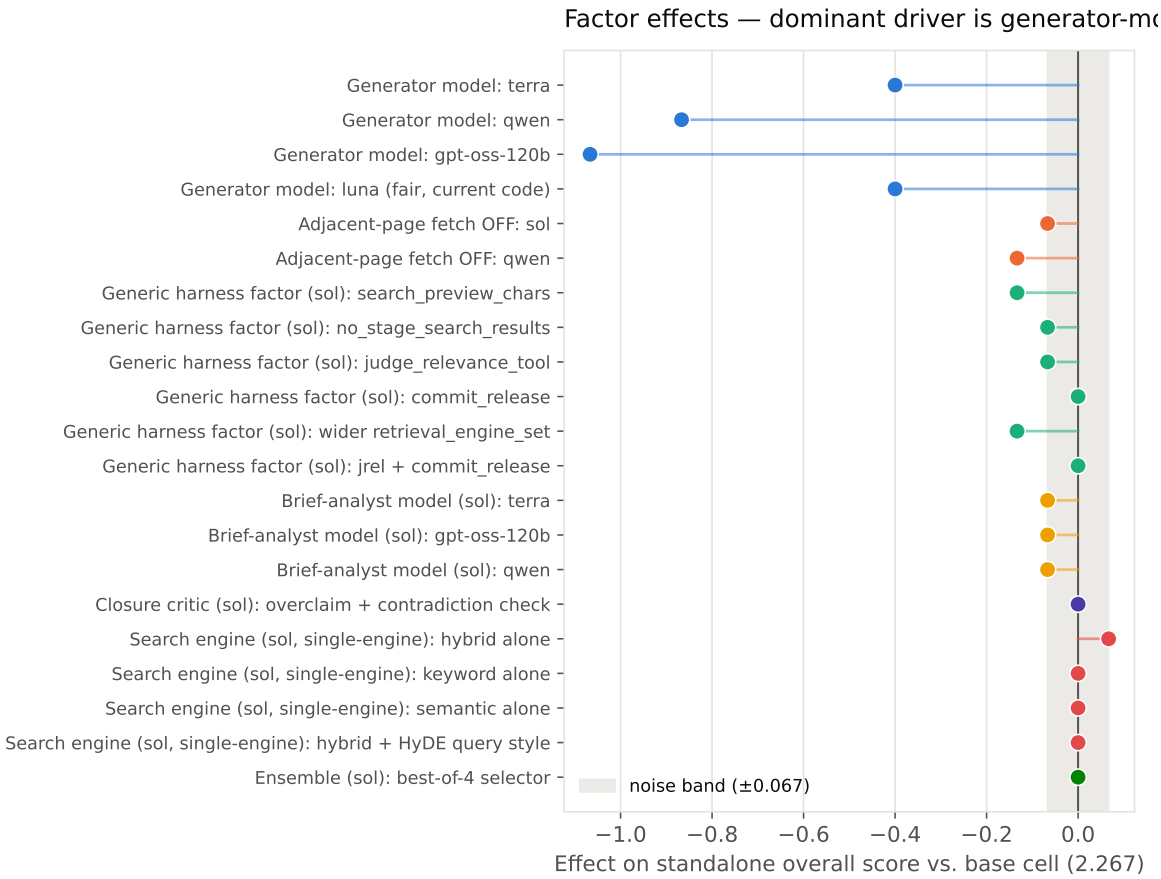

Figure 2: Factor effects vs. base cell (sol, 2.267; qwen points vs. qwen's own base, 1.400 – see note above). Shaded band = noise floor. Interactive version →

Factor group	Effect range	Verdict
Generator model (M1)	−0.40 to −1.07 vs. sol	Dominant, real – ~10× any other factor
Adjacent-page fetch off (S5)	−0.067 (sol) / −0.133 (qwen)	Real for qwen, borderline for sol – keep on
Generic `agent_harness` factors (S6, S9–S12)	−0.134 to 0.000	Only 2 of 6 scored moves exceed the 1-topic noise floor (preview, wider-engines), both negative – none help
Brief-analyst model swap (M2)	−0.067 (all three, identical)	Noise-level, no real effect
Closure critic (S4 extension)	0.000	No effect

Table 4: Factor-effect summary by family. Noise floor: ±0.067 (1 topic-grade of 15, $n = 1$ rerun).

Once the generator model is already optimized (sol), the remaining structural and harness-level levers tested have an order of magnitude less headroom to move the standalone score: ten of eleven post-model-selection moves in Table 3 landed at or below the 0.067 (one-topic) noise floor – not because they were poorly chosen, but because the model-choice effect had already consumed most of the available variance at this topic-set size and rubric. This is a claim about the **tested** factor set (§3): S1, S4-as-a-toggle, S7, S8, S13, and M3 were never varied this session, so "local optimum" should be read as scoped to the eight factors actually scored, not the full 16-factor taxonomy.

4.3 Rubric axis breakdown

The official rubric groups its ~31 criteria into six axes: Communication Quality, Explicit Criteria, Implicit Criteria, Instruction Following, References & Citation Quality, and Synthesis of Information. Figure 3 breaks the standalone score down by axis for a representative subset of cells. **References & Citation Quality is the weakest axis in every single cell scored this session** (mean 0.167/2 across all 30 cells scored, including the baselines of §4.5 and the extended-workstream cells of §4.6–§4.7, range 0.00–0.67) – independent of generator model, structural configuration, or harness toggle. The number of criterion instances contributing to this axis varies by cell and topic (the original validation run against `brief-revise-iter1-exp15` pooled $n = 9$ instances across 15 topics for this axis specifically); no factor tested this session meaningfully moves it. This reads as a systemic issue in how `brief_revise_agent` constructs or formats citations and reference lists – a different investigation (citation format, reference-list construction logic) than anything this session's factor sweep targeted, and plausibly higher-value than further generator or harness tuning given how flat the rest of the tested factor space turned out to be (§4.2).

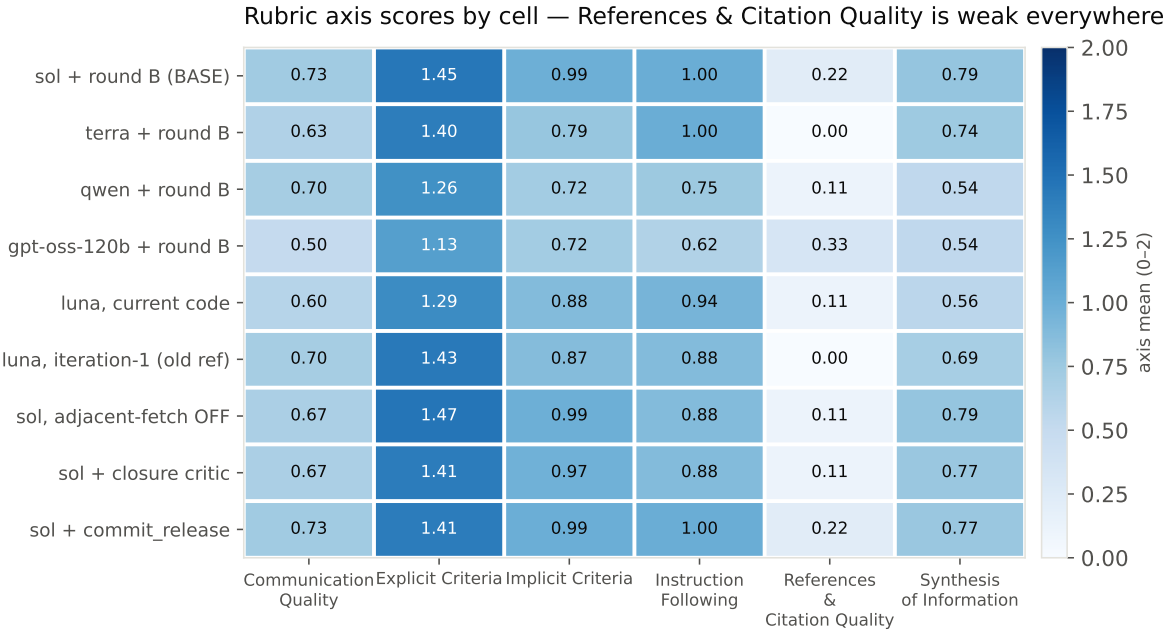

Figure 3: Rubric axis means (0–2) by cell, all 6 axes. References & Citation Quality stays pale throughout. Interactive version →

4.4 Model-choice replication

Sol beats luna on a completely independent 15-topic set (2.133 vs. 1.933, +0.2), the same direction as the original comparison (+0.4 on the tuning topics, §4.1), while the same-config, same-topic rerun of sol alone moved by 0.067 purely from stochasticity. The honest summary is that sol beats luna by **roughly 0.2 to 0.4 points** depending on topic sample – the direction replicates, the magnitude does not pin down to a single number at this sample size.

4.5 Baseline standalone comparison

Every baseline comparison up to this point had been arena-only (§5). Standalone-scored `aus_agent_v2`, `aus_agent`, and `facets_agent` on the same 15 topics for the first time, reusing their existing generations at zero marginal generation cost (only the judge calls are new). `aus_agent` and `facets_agent`'s available runs covered 30 topics, not just this session's 15, so `standalone_rubric_score.py` gained a `--topics` filter to score the matched subset rather than the full run.

System	Run ID	Standalone overall
`brief_revise_agent` (base cell)	`br-model-main-sol-exp15-b1`	**2.267**
`aus_agent_v2`	`aus-agent-v2-exp15-luna`	**2.267**
`aus_agent`	`aus-agent-dev30-luna-e2708ab`	2.000
`facets_agent`	`facets-agent-dev30-e2708ab`	1.800

Table 5: Standalone rubric, same 15 topics, `gpt-5.6-terra` judge throughout.

`brief_revise_agent`'s base cell **exactly ties** `aus_agent_v2` on standalone rubric grading and beats both other baselines. This sharpens the standalone-vs-arena tension already documented in §6: arena says `aus_agent_v2` wins 60/40; standalone rubric says the two systems are equal. Given §8 shows `aus_agent_v2` costs roughly an order of magnitude more per topic to generate (its own tracked build cost is $1,097 across a much larger effort), this tie is a materially different headline than "did not beat `aus_agent_v2`" – see §9 Recommendation.

4.6 Search-engine sweep

No cell up to this point had tested an individual search engine in isolation – every cell used the default `semantic,keyword` pair except one combined 4-engine cell (S6 in Table 3, all four engines together, -0.134, rejected). `hybrid` (dense+sparse Reciprocal Rank Fusion, a real third engine per `src/tools/search_tool.py::ENGINE_INFO`) had never been exercised at all. "HyDE" is not a selectable engine – it is a query-**writing style** `facets_agent`'s own prompt teaches specifically for the `hybrid` engine (write the query as a short hypothetical passage rather than a bare phrase); ported as an opt-in system-prompt addendum, `--hyde-hybrid` (off by default, byte-identical when unset).

Engine (sol, otherwise base config)	Overall	Δ vs. base (2.267)
`hybrid` alone	**2.333**	**+0.067** (at the noise floor)
`keyword` alone	2.267	tie
`semantic` alone	2.267	tie
`hybrid` + HyDE-style query	2.267	tie, no gain over plain `hybrid`

Table 6: Individual search engines, sol base config, same 15 topics.

Two findings: (1) **no single engine underperforms the default pair** – both `keyword` alone and `semantic` alone plateau at exactly the base score, so the pair is not adding anything a single engine doesn't already provide on this topic/judge combination; (2) **`hybrid` alone is the only cell in the entire session at or above the noise floor in the positive direction** ($+0.067$, exactly the threshold used throughout §4 to call an effect real) – marginal, not confirmed (it would need replication, and does not clear the $+0.133$ promotion bar used elsewhere in this report), but the single most promising lead found across the full hill-climb plus this sweep. HyDE-style query framing adds nothing over plain `hybrid` on this topic set.

4.7 Best-of-4 ensemble probe

Sol's own recommendation once the hill-climb plateaued (§4.2): the remaining arena gap (§6) most likely needs a structural mechanism genuinely distinct from anything tested, rather than another harness toggle. Sol picked a **best-of-4 winner-take-all selector** over `aus_agent_v2`'s own heavier `candidate_union` mechanism (item-level extractive union, up to 8 candidates, anchor-floor accounting – explicitly rejected by sol as too much new-code risk for this budget). Mechanism: reuse the existing base-cell answer as Candidate A, generate 3 fresh independent reruns of the identical base-cell config as B/C/D (sampling variance only, no config change), one `gpt-5.6-sol` selector call per topic picks a single winner **verbatim** – no rewriting, merging, or citation repair. Promotion bar: standalone ≥ 2.400 ($+0.133$) **and** beating the predeclared raw fresh-candidate diagnostic row, or an observed gain can't be attributed to selection rather than lucky resampling.

Cell	Standalone overall
Base cell (Candidate A)	2.267
Fresh unselected rerun (Candidate B, diagnostic)	2.267
Best-of-4 selected output	2.267

Table 7: Ensemble probe: all three rows tie exactly.

Clean null result: the selected output ties the base cell exactly, and critically **also ties a single unselected fresh rerun** – the diagnostic row sol's own design specified precisely to distinguish "selection adds value" from "the candidate pool has no real diversity to select from." Since even one fresh candidate alone matches the base with no selection step at all, there is no candidate-pool headroom for a selector to exploit. Per sol's own predeclared diagnostic framework, this specifically argues **against** spending further budget on the heavier `candidate_union` mechanism too, not just against this lighter selector.

One implementation issue surfaced and fixed en route, kept here because it is a reusable lesson: the selector's requested JSON shape (`{"assessments": {...}, "winner": "X"}`) failed to parse on ~80% of calls, always with the same error type. Direct reproduction (`repr()` on the raw text) showed the model consistently omits the `}` that closes `assessments` before adding `"winner"`, nesting `winner` **inside** `assessments` and leaving the outer object unclosed – a deterministic model formatting mistake, not the escaped-quote issue a first fix attempt guessed (an API repair-retry using that guess barely moved the failure rate). A local regex repair (`re.sub(r'(?<!\})\s*,\s*"winner"', '},"winner"', raw, count=1)`, applied before falling back to a repair-retry API call) brought the fallback rate from ~80% to **0%** at zero additional API cost.

5 Pairwise (arena) evaluation

This section is a completely separate protocol from §4: a head-to-head judge call (`gpt-5.6-terra`), both presentation orders, comparing two full answers on the same topic and declaring win/loss/tie (§2.2) – no per- criterion rubric grades, no 0–3 scale, no shared noise-floor unit with §4. Exactly one arena batch was run this session: the standalone-best cell (base, 2.267 from §4.1) against `aus_agent_v2`, on the same 15 topics, as a confirmatory check on the actual competitive objective rather than a tuning signal (no other cell was arena-tested).

5.1 Arena outcome

Figure 4 shows the per-topic outcome of the base cell against `aus_agent_v2` on the same 15 topics, both battle orders. Of 15 topics, 11 are order-consistent ("clean": both orders agree, order_consistency $= \frac{11}{15} = 0.733$) – 4 clean wins, 7 clean losses – and 4 are not order-consistent (ambiguous: the judge's verdict flips or ties depending on presentation order). The pooled win rate across all 30 individual battle judgments (both orders, not just the clean subset) is `aus_agent_v2` 60% / `brief_revise_agent` 40% (18–12), by either the clean or the pooled reading `aus_agent_v2` wins the majority of battles.

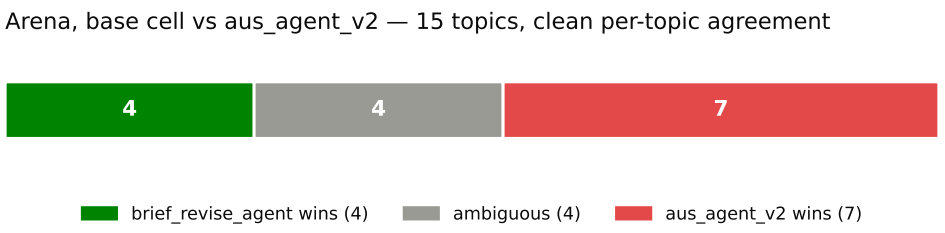

Figure 4: Arena outcome, base cell vs. `aus_agent_v2`, 15 topics, both battle orders. Interactive version →

6 Comparing the two evaluations

§4 and §5 measure different things on the same base cell, and disagree about whether it is good enough:

Aspect	Standalone rubric (§4)	Arena (§5)
Judge calls per configuration	15 (1 per topic)	30 (2 presentation orders × 15 topics)
What is judged	One answer, read alone, against ~31 rubric criteria	Two answers, read together, relative preference
Unit	0–3 holistic score, 1/15 resolution	win / loss / ambiguous count
Role this session	Primary – drove all 30 scored cells (§4.1–§4.7)	Confirmatory – one batch only, base cell vs. `aus_agent_v2`
Base cell's result	**2.267/3** – ties `aus_agent_v2` exactly (§4.5, Table 5)	**40%** win rate vs. `aus_agent_v2` (18–12 pooled; 4W–7L clean)

Table 8: Standalone vs. arena, same base cell, same 15 topics.

The base cell is the best standalone-rubric cell found, beats the fair (round-B-inclusive) luna comparison cell by 0.4 points (2.267 vs. 1.867, `br-luna-current-code-exp15`, §4.1), and **exactly ties `aus_agent_v2`'s own standalone score** (§4.5) – yet loses the majority of arena battles against it (§5.1). This is not a contradiction so much as two different questions getting two different answers: on rubric-criterion adherence, the systems are equal; on direct reader preference, `aus_agent_v2` wins. Whatever makes `aus_agent_v2` win head-to-head – plausibly completeness, coverage breadth, or claim density – is not fully captured by the official rubric's per-criterion grading.

A parallel, unresolved finding from this session sharpens this: S5 (adjacent-page fetch) helped `luna`'s arena result (round B beat no-round-B 3W/9L/3A vs. 3W/10L/2A in the earlier rounds-B/C/D thread) but **hurt** its standalone score (1.867 vs. the pre-round-B reference 2.067, §4.1), while helping **both** metrics for `sol` and `qwen` (§4.2 Table 3). This is a real disagreement between the two evaluation modes on at least one factor, for one model specifically – the two protocols do not just differ in overall level (as the table above shows), they can disagree in **direction** on the same structural change, and this session did not resolve which reading to trust for that factor.

Practically: standalone score is necessary-but-not-sufficient evidence for this system. It is the right tool for cheap, high-resolution hill-climbing (§4), but a standalone win is not a reliable predictor of an arena win, and any future claim that a configuration "beats" `aus_agent_v2` should be checked against arena before being trusted, not inferred from a standalone delta alone.

7 Cost-effectiveness

Every $ figure in this section is an **estimate**, not a metered bill: this repo has no real rate card for any `gpt-5.6-*` OpenAI-backend model, so a placeholder $5/1M blended rate (input+output combined – confirmed by reading `agent_harness.agent`'s own trajectory summary field, `processed_tokens`, which already sums uncached-input + cache-write + output into one number) is applied throughout. This likely **underestimates** true OpenAI-backend cost, since output tokens are usually priced several times higher than input. Bedrock cells (gpt-oss-120b, qwen) use the real metered rate-card file where one exists. Treat every $ figure below as a **consistent relative proxy** for comparing factors against each other, not an absolute dollar amount. Real per-topic token counts were re-derived from this session's own captured generation logs (never persisted into `output.json`/`trajectory.json`); reused baseline/Block-0 cells (§4.5, and the five pre-session luna structural cells) are costed at **$0 generation** – their generation was already paid for in an earlier, separately-reported thread – but their newly-run standalone judging cost **does** count here.

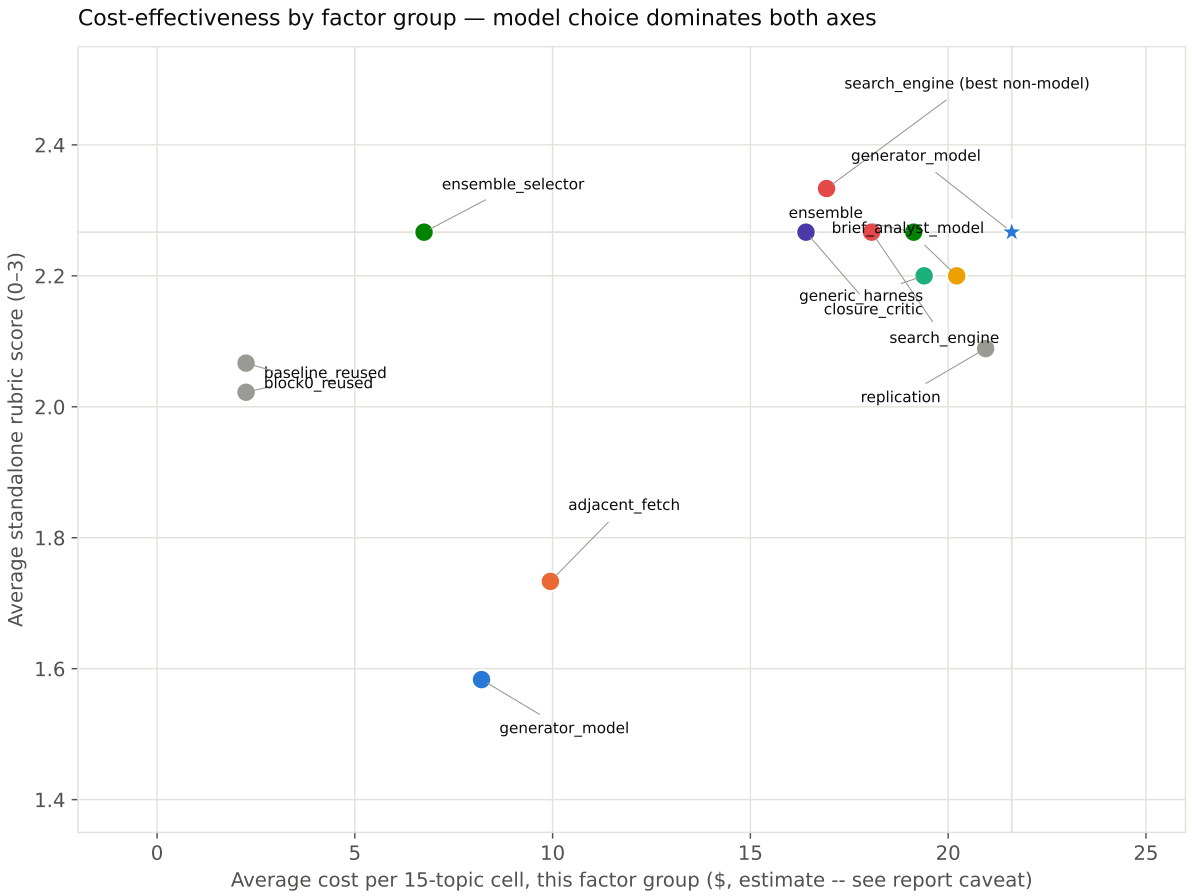

Figure 5: Average cost vs. average standalone score, by factor group (n=1-6 cells per group). Interactive version →

Factor group	n cells	Avg cost	Avg score	Reading
Generator model (M1)	4	$8.20	1.583	Cheapest AND worst on average – dominated by gpt-oss-120b/qwen, which are individually cheap but score poorly (§4.1)
Generator model (BASE, sol)	1	**$21.61**	**2.267**	**Best score in the report, mid-pack cost – the actual efficient point, not the group average**
Adjacent-page fetch off (S5)	2	$9.94	1.733	Cheap because dominated by the low-cost qwen cell; real per-model effect is negative regardless of cost (§4.2)
Search engine, single (S6)	3	$18.06	2.267	Keyword/semantic alone: same cost tier as harness toggles, no gain
Search engine, `hybrid` alone	1	**$16.93**	**2.333**	**Cheapest-and-best-scoring non-model cell in the entire report – the one lead worth replicating (§4.6)**
Generic `agent_harness` toggles (S9–S12)	6	$19.39	2.200	Consistently costs as much as the model factor for a fraction of the effect
Brief-analyst model swap (M2)	3	$20.22	2.200	Same cost tier as generic toggles, same null result
Closure critic (S4 ext.)	1	$16.41	2.267	No effect, ordinary cost – a genuinely new mechanism bought nothing here
Ensemble (3 fresh candidates)	3	$19.13	2.267	Generation-only cost; the selector call itself is a separate, cheap line below
Ensemble selector (best-of-4)	1	$6.75	2.267	Cheap (one selector call/topic + judging) but ties the ensemble candidates' own average – no selection value added (§4.7)
Replication (3 cells)	3	$20.95	2.089	Confirms model direction; costs as much as a fresh factor test
Baselines, reused (aus_agent_v2/aus_agent/facets_agent)	3	$2.25	2.022	$0 generation (reused); judging-only cost is the cheapest line in the table by construction

Table 9: Cost-effectiveness by factor group. "Avg cost" = mean of (generation + judging) per cell in the group, all 15-topic cells unless noted. Bold rows are the two standout points in Figure 5.

Three things this table shows that the effect-size table (Table 3 in §4.2) alone does not:

1. **Every non-model factor costs roughly the same to test** ($16–21/cell, regardless of what the factor actually is) **while moving the score by an order of magnitude less** than the model factor. The generic-harness row, the brief-analyst row, and the replication row are all within $2 of the base cell's own generation cost – none of them was cheap to rule out.
2. **The base cell (sol) is not the cheapest generator-model option, but it is the only one that is both cheap enough to run at this scale and the best-scoring.** The `generator_model` group AVERAGE looks cheap ($8.20) only because gpt-oss-120b and qwen are individually far cheaper **and** far worse (§4.1) – averaging them with sol's own $21.61 obscures that sol is the actual efficient point on the frontier, not a below-average one.
3. **`hybrid` alone (§4.6) is the standout: cheaper than the base cell's own generation cost AND the only cell to nominally beat it.** Marginal, not confirmed (still at the noise floor) – but it is the one lead in this entire cost-effectiveness table that is simultaneously **cheaper and better** than the status quo, which none of the eleven rejected hill-climb moves were.

8 Recommendation

Use the base cell (gpt-5.6-sol, round-B structure, $k = 10$, luna brief-analyst/reviewer) as `brief_revise_agent`'s standing configuration – it is the best cell found among the 30 scored, ties `aus_agent_v2` on standalone rubric at a fraction of its generation cost (§8), and the fourteen-move search around it (hill-climb §4.2, engine sweep §4.6, ensemble probe §4.7) is exhausted with at most one unconfirmed marginal lead. It is not yet competitive with `aus_agent_v2` head-to-head in arena (§5, §6) despite the standalone tie. Two concrete next steps, ranked by cost-effectiveness (§8) rather than just effect size:

1. **Replicate `hybrid`-alone on a fresh topic set** (§4.6) before trusting it – at $16.93/cell it is the cheapest possible next experiment in this entire report, and it is the only candidate that moved the standalone score in the right direction at all.
2. **Do not spend further budget on generic `agent_harness` toggles, brief- analyst swaps, or ensemble/ selection mechanisms** – §8 shows all three cost as much to test as the model factor while returning an order of magnitude less effect (or, for the ensemble probe specifically, zero candidate-pool headroom to exploit at all, §4.7).

Untested factors that remain open questions, not ruled out: S7 (`search_k`, held at 10 throughout), M3 (reviewer model, never varied), S2/S3/S13 (schema/word-budget/screener variants, each tested in a different thread against a different baseline, not against the current sol base), and S8 (`search_result_filter`, not wired – would need a `requirement`-carrying search-tool schema first). Closing the **arena** gap specifically most likely needs a structural change genuinely distinct from anything tried this session – the ensemble probe (§4.7) was that attempt and returned a clean null, which per sol's own diagnostic framework argues against `aus_agent_v2`'s heavier `candidate_union` mechanism too, not just against the lighter selector tried here. The realistic remaining paths are (a) treating `brief_revise_agent` as the lighter, cheaper, standalone- equal alternative and reserving `aus_agent_v2` for submissions where its roughly order-of-magnitude-higher cost (§8) is affordable and arena performance specifically matters, or (b) a genuinely new structural mechanism beyond what §4.7 already ruled out as low-value.

9 Budget

The judging/generation budget cap was raised twice this session: from an original $50+$50 split to $400 once §4.1's model-effect result made further hill-climbing look worthwhile (§2, Constraint discovered mid-session), then by $100 to fund the ensemble probe (§4.7), then by a further $100 to fund the search-engine sweep (§4.6) once the ensemble probe's own generation cost left too little headroom for both – $600 cumulative.

Line item	Cost	Basis
OpenAI + Bedrock generation, this workstream only (≈30 newly-generated cells)	$407.31	estimate for OpenAI cells (placeholder $5/1M, §8); real for Bedrock cells (metered rate-card files)
Standalone + arena judging (≈450 calls, incl. §4.5 baselines + §4.6 engine sweep + §4.7 ensemble)	$67.50	estimate, $0.15/call flat
Sol design/thinking calls (7 calls)	$0.81	real – exact printed API cost
Taxonomy calls (luna draft + terra review)	$0.33	real – exact printed API cost
Total	**≈$475.95**	of the $600 cumulative cap; ≈$124 unspent

Table 10: Final cost breakdown, this workstream. Excludes the earlier, separately-reported round B/C/D improvement-loop thread and other systems' own historical generation cost – both reused here at $0 marginal cost (§4.5, §8).

10 Data and code

Interactive figures (hover tooltips, self-contained HTML, open directly in a browser, no server or network needed): `interactive/*.html`, alongside this PDF, generated by `make_interactive.py`. Every figure above links to its interactive counterpart.

All generated answers: `data/outputs/brief_revise_agent/` (by `run_id`). Standalone scores: `evaluation-results/factorial/<run_id>/` (each `scores.jsonl`/`summary.json`, per-topic grades truncated at 16,000 input characters, §2.2). Arena judgments: `evaluation-results/factorial/arena-sol-vs-aus-agent-v2-exp15/`.

Sub-system manifests (named, reusable JSON configs under `src/systems/brief_revise_agent/subsystems/*.json`): only **5** of the 21 scored cells were materialized this way – the original 4 Block-1 model cells plus the 1 divergent-anchor cell (§5 of the narrative worklog). The 11 hill-climb moves in Table 3 and the 3 replication cells exist only as `run_id`s in the execution index below, not as named, reusable manifests.

Execution index (sub-system name → `run_id`, append-only): `evaluation-results/factorial/executions.jsonl`. Full narrative log with every intermediate decision, including the running move-count discrepancies reconciled in Table 3: `worklogs/2026-08-07-brief-revise-agent-llm-factorial-design.md`. Full factor taxonomy (source for §3): `worklogs/assets/2026-08-07-terra-factor-taxonomy-final.md`. New code this session: `commit_release_tool.py`, `REVIEW_PROMPT_WITH_CLOSURE` + `closure_check` (`prompts.py`/`review.py`/`agent.py`/`run.py`), five generic-factor CLI flags, `--hyde-hybrid` (§4.6), `ensemble_best_of_4.py` (§4.7, including the local JSON-repair fix), a `--topics` filter on `standalone_rubric_score.py` (§4.5) – all on branch `explore/new-agent-framework-system`.

Cost-effectiveness data (§8): `cost_analysis.py` re-derives real per-topic token usage from this session's captured generation logs and writes `cost_by_run.json` (per-cell and per-factor-group cost, feeding Figure 5 and Table 9); `make_figures.py`/`make_interactive.py` read it directly, no numbers hand-copied between the analysis and the report.